SIMATS ENGINEERING
ASSESSMENT TOOL 3

Course Name	INTERNET PROGRAMMING
Course Code	CSA43
Name	B KIRTHANAA
Register Number	192324244
Course Outcome Covered	CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)
Assessment Tool	Short Answer Test

Question 1: Responsive Web Design

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

SOLUTION:

(a) Concept of responsive web design and its importance

Responsive web design (RWD) is an approach in which one website adapts its layout, images and text to the screen size, resolution and orientation of the device. It is built on three ideas: flexible grids (layouts in relative units), flexible media (images that scale) and CSS media queries (rules that apply only at certain screen widths). The company does not need separate sites for mobile, tablet and desktop.

- One code base: a single site is cheaper to build, test and maintain than separate mobile and desktop versions.
- Better user experience: visitors can read and tap everything without pinch-zooming or scrolling sideways.
- Wider reach: a large share of web traffic comes from phones, so a non-responsive site loses customers.
- Search ranking: search engines index the mobile version first and prefer one URL for all devices.
- Future-proof: the same design works on new screen sizes, such as foldables and large monitors.

(b) Key CSS techniques used to achieve responsiveness

Technique	How it helps
Viewport meta tag	<meta name="viewport" content="width=device-width, initial-scale=1.0"> makes the page width equal to the device width instead of a zoomed-out desktop width.
Fluid units	Percentages, fr, vw, rem and em replace fixed pixels, so boxes and text grow and shrink with the screen.

Technique	How it helps
Flexible images	max-width: 100%; height: auto; stops images from overflowing their container.
Media queries	@media (min-width: 768px) { ... } applies extra rules from a breakpoint upwards. Writing the base CSS for phones and adding min-width queries is the mobile-first approach.
Responsive typography	clamp(1.5rem, 5vw, 2.5rem) keeps headings readable on every screen.
Modern layout modules	Flexbox and Grid rearrange, wrap and resize content without floats or fixed widths.

(c) Using Flexbox or Grid to design adaptive layouts

- Flexbox (one-dimensional): display: flex with flex-wrap: wrap and flex: 1 1 220px lets items share a row and drop to the next row when space runs out. Changing flex-direction from column to row inside a media query turns a stacked mobile menu into a horizontal desktop menu.
- Grid (two-dimensional): grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)) creates as many columns as fit the width, so the card layout adapts without any media query. grid-template-areas can also be redefined at a breakpoint to move whole sections.

(d) Example layout

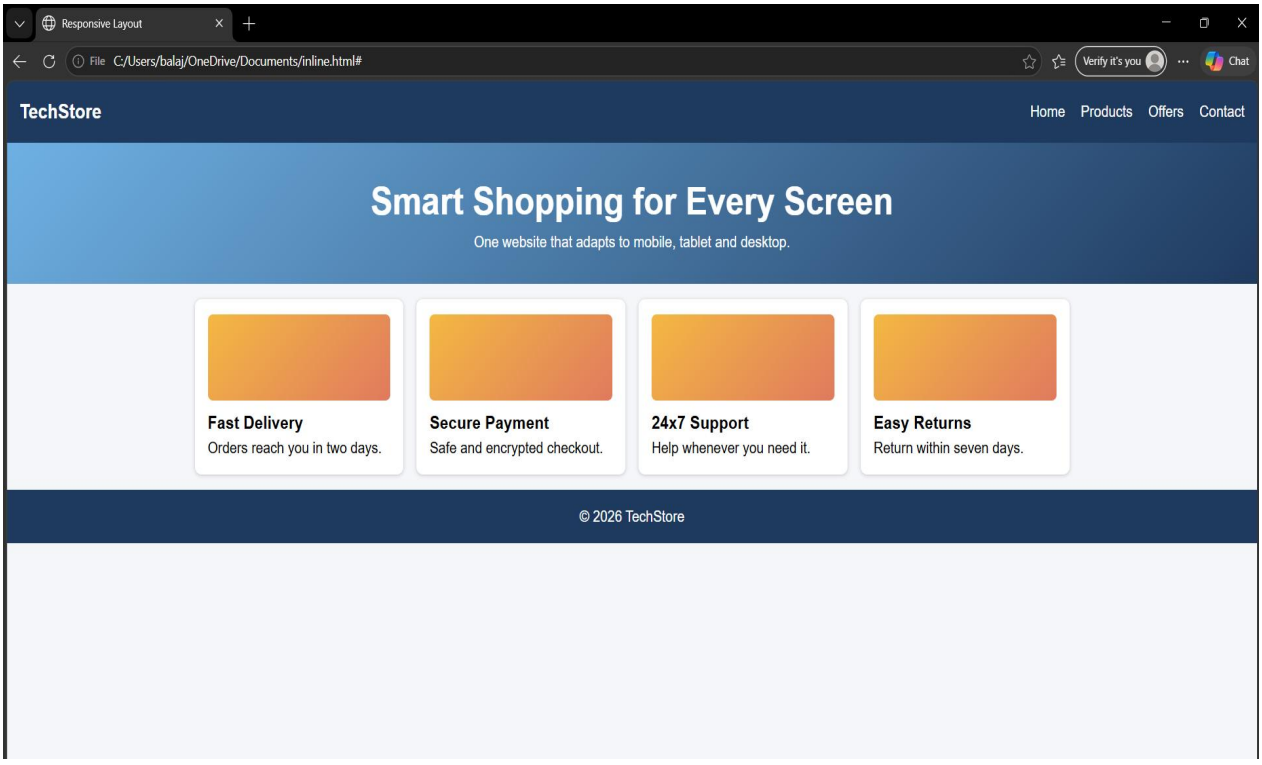
The page below uses Flexbox for the header and navigation, and Grid for the cards. The navigation is stacked on phones and becomes a row from 768px.

HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Layout</title>
<style>
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: Arial, sans-serif; line-height: 1.5; background: #f4f6fa; }
header { background: #1f3a5f; color: #fff; padding: 0.8rem 1rem;
         display: flex; flex-wrap: wrap; justify-content: space-between;
         align-items: center; }
.logo { font-size: 1.3rem; font-weight: bold; }
nav { display: flex; flex-direction: column; width: 100%; }   /* mobile: stacked */
nav a { color: #fff; text-decoration: none; padding: 0.5rem 0; }
.hero { padding: 2rem 1rem; text-align: center; color: #fff;
        background: linear-gradient(135deg, #6fb1e4, #1f3a5f); }
.hero h1 { font-size: clamp(1.5rem, 5vw, 2.5rem); }
.grid { display: grid; gap: 1rem; padding: 1rem; max-width: 1100px; margin: 0 auto;
        grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
.card { background: #fff; border-radius: 8px; padding: 1rem;
        box-shadow: 0 1px 4px rgba(0,0,0,.2); }
.pic { height: 90px; border-radius: 6px; margin-bottom: 0.6rem;
       background: linear-gradient(135deg, #f4b942, #e07a5f); }
footer { background: #1f3a5f; color: #fff; text-align: center; padding: 1rem; }
@media (min-width: 768px) {          /* tablet and desktop */
  nav { flex-direction: row; width: auto; gap: 1.2rem; }
}
</style>
```

```
</head>
<body>
<header>
 <div class="logo">TechStore</div>
 <nav>
  <a href="#">Home</a><a href="#">Products</a>
  <a href="#">Offers</a><a href="#">Contact</a>
 </nav>
</header>
<section class="hero">
 <h1>Smart Shopping for Every Screen</h1>
 <p>One website that adapts to mobile, tablet and desktop.</p>
</section>
<main class="grid">
 <div class="card"><div class="pic"></div>
  <h3>Fast Delivery</h3><p>Orders reach you in two days.</p></div>
 <div class="card"><div class="pic"></div>
  <h3>Secure Payment</h3><p>Safe and encrypted checkout.</p></div>
 <div class="card"><div class="pic"></div>
  <h3>24x7 Support</h3><p>Help whenever you need it.</p></div>
 <div class="card"><div class="pic"></div>
  <h3>Easy Returns</h3><p>Return within seven days.</p></div>
</main>
<footer>&copy; 2026 TechStore</footer>
</body>
</html>
```

OUTPUT:



Desktop: navigation in one row, four cards in one row

(e) Improvements to enhance user experience across devices

- Use a hamburger menu and a sticky header on phones, and keep every tap target at least 44px high.
- Serve responsive images with srcset and <picture>, compressed formats such as WebP, and loading="lazy".

- Keep line length between 60 and 75 characters and use at least 16px body text for comfortable reading.
- Support dark mode with prefers-color-scheme and reduced motion with prefers-reduced-motion.
- Add alt text, visible focus styles and sufficient colour contrast for accessibility.
- Test on real devices and in browser developer tools, and set breakpoints where the content starts to break, not at fixed device sizes.

Question 2: Client-Side Form Validation

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

SOLUTION:

(a) Concept and purpose of client-side validation

Client-side validation means checking the data a user types in the browser, using JavaScript or HTML5 attributes, before the form is sent to the server. Its purposes are:

- Instant feedback: the user sees the error at once instead of after a page reload.
- Fewer server requests: obviously wrong data never leaves the browser, which saves bandwidth and server load.
- Cleaner data: required fields, formats and lengths are checked before they reach the database.
- Guidance: clear messages tell the user exactly how to fix each field.

Client-side validation is a convenience for the user. It is not a security measure, because a user can bypass it, so the server must validate the data again.

(b) JavaScript event handling in validation

Validation is attached to form elements with addEventListener(event, handler). When the event fires, the handler reads input.value, tests it, writes the error text into the page (DOM), and returns true or false.

Event	Use in validation
submit	Runs when the form is submitted. All fields are checked, and e.preventDefault() stops the submission if any field is invalid.
blur	Runs when the user leaves a field, so that field is validated without interrupting typing.
input / keyup	Runs on every keystroke, so an existing error disappears as soon as the value becomes valid.
change / focus	change validates select boxes, checkboxes and radio buttons, and focus can show a format hint.

(c) Role of regular expressions

A regular expression (regex) is a pattern that describes the allowed format of a string. In JavaScript, pattern.test(value) returns true when the text matches. Regex checks the structure of input, such as the position of "@" in an email or the number of digits in a phone number, in a single line of code.

Input	Regular expression	Meaning
Name	/^[A-Za-z]{3,}$/	At least 3 characters, only letters and spaces.
Email	/^[^\s@]+@[^\s@]+\.[A-Za-z]{2,}$/	Some text without spaces or "@", then "@", then a domain, a dot and at least 2 letters.
Mobile (India)	/^[6-9]\d{9}$/	The first digit is 6 to 9, followed by exactly 9 more digits (10 digits in total).

The symbols ^ and $ force the whole string to match. A regex checks only the format, so it cannot prove that an email address really exists.

(d) Example

The registration form validates each field when the user leaves it, re-checks all fields on submit, shows an error under each invalid field and sets aria-invalid for screen readers.

HTML, CSS & JAVASCRIPT CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Form Validation</title>
<style>
* { box-sizing: border-box; }
body { font-family: Arial, sans-serif; background: #eef2f7; display: flex;
      justify-content: center; padding: 30px; }
form { background: #fff; padding: 20px; width: 340px; border-radius: 8px;
      box-shadow: 0 1px 4px rgba(0,0,0,.2); }
h2 { color: #1f3a5f; margin-bottom: 10px; }
label { display: block; margin-top: 12px; font-weight: bold; }
input { width: 100%; padding: 8px; margin-top: 4px; border: 1px solid #999;
       border-radius: 4px; }
input.invalid { border-color: #c0392b; background: #fdecea; }
.error { color: #c0392b; font-size: 0.85rem; display: block; min-height: 1.1em; }
button { margin-top: 14px; width: 100%; padding: 10px; border: 0;
        border-radius: 4px;
        background: #1f3a5f; color: #fff; font-size: 1rem; cursor: pointer; }
#msg { margin-top: 12px; font-weight: bold; }
.ok { color: #1e8449; } .bad { color: #c0392b; }
</style>
</head>
<body>
<form id="regForm" novalidate>
 <h2>Registration</h2>
 <label for="name">Full Name</label>
 <input type="text" id="name" autocomplete="name">
 <span class="error" id="nameErr" aria-live="polite"></span>

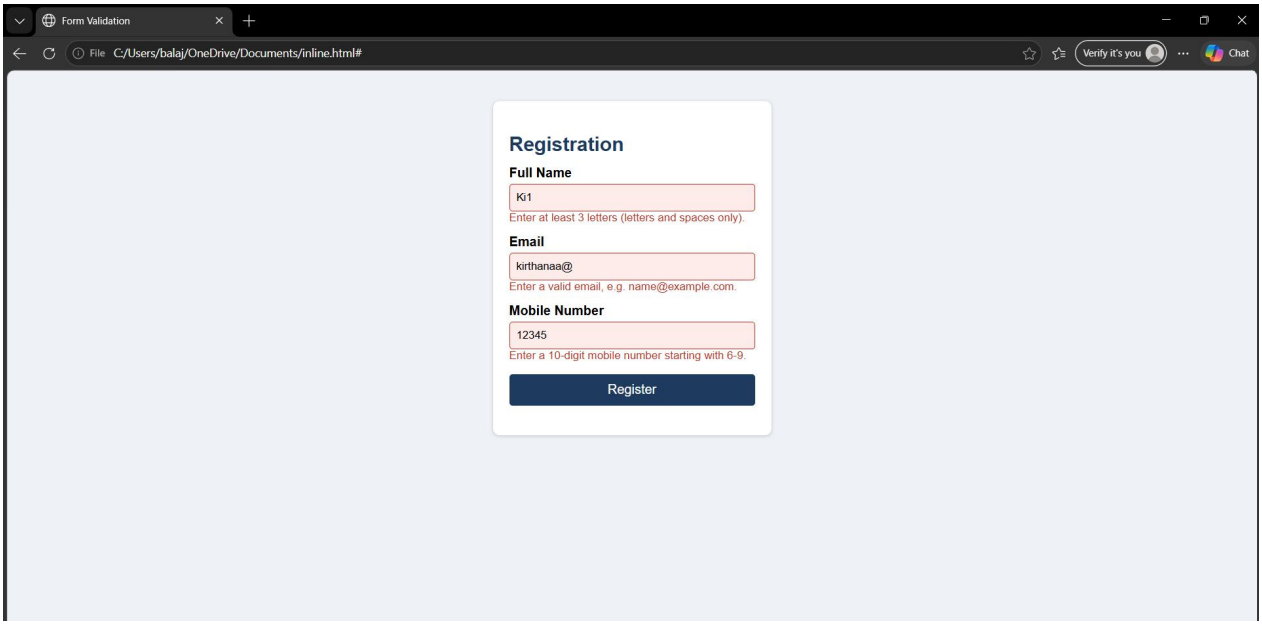
 <label for="email">Email</label>
 <input type="email" id="email" autocomplete="email">
```

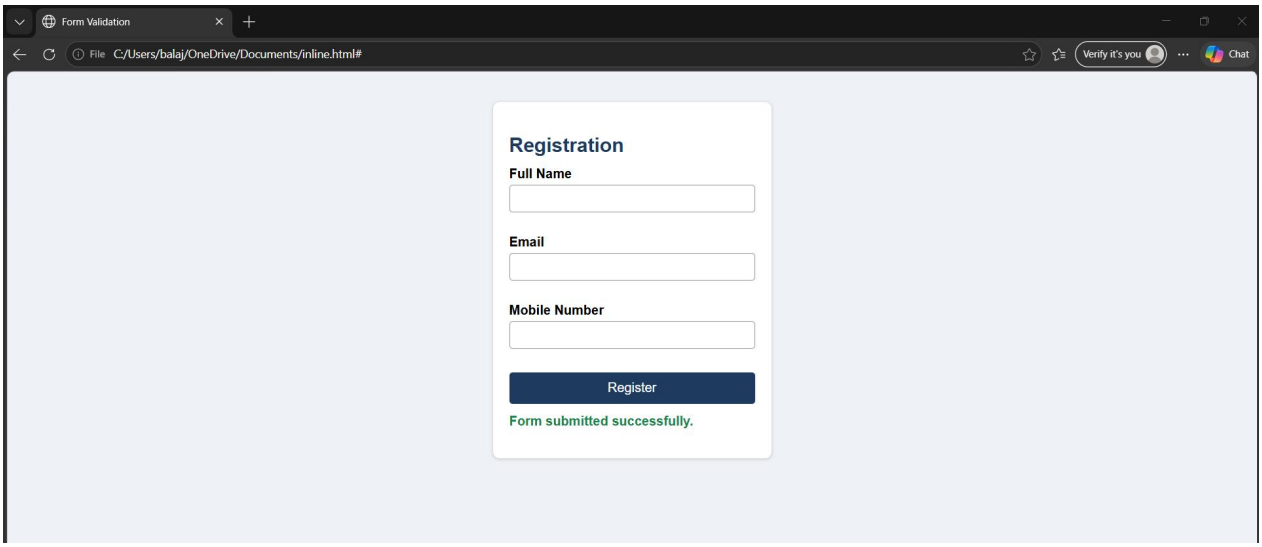
```
  <span class="error" id="emailErr" aria-live="polite"></span>

  <label for="phone">Mobile Number</label>
  <input type="tel" id="phone" inputmode="numeric" maxlength="10">
  <span class="error" id="phoneErr" aria-live="polite"></span>

  <button type="submit">Register</button>
  <p id="msg" aria-live="polite"></p>
</form>
<script>
const patterns = {
  name:  /^[A-Za-z ]{3,}$/,
  email: /^[^\s@]+@[^\s@]+\.[A-Za-z]{2,}$/,
  phone: /^[6-9]\d{9}$/
};
const messages = {
  name:  "Enter at least 3 letters (letters and spaces only).",
  email: "Enter a valid email, e.g. name@example.com.",
  phone: "Enter a 10-digit mobile number starting with 6-9."
};
function validateField(id) {
  const input = document.getElementById(id);
  const ok = patterns[id].test(input.value.trim());
  document.getElementById(id + "Err").textContent = ok ? "" : messages[id];
  input.classList.toggle("invalid", !ok);
  input.setAttribute("aria-invalid", !ok);
  return ok;
}
Object.keys(patterns).forEach(function (id) {
  const input = document.getElementById(id);
  input.addEventListener("blur", function () { validateField(id); });
  input.addEventListener("input", function () {
    if (input.classList.contains("invalid")) validateField(id);
  });
});
document.getElementById("regForm").addEventListener("submit", function (e) {
  e.preventDefault();                       // stop the page from reloading
  const results = Object.keys(patterns).map(validateField);
  const msg = document.getElementById("msg");
  if (results.every(Boolean)) {
    msg.textContent = "Form submitted successfully.";
    msg.className = "ok";
    this.reset();
  } else {
    msg.textContent = "Please correct the errors above.";
    msg.className = "bad";
  }
});
</script>
</body>
</html>
```

OUTPUT:





(e) Improvements for better usability and security

Usability

- Show the message next to the field, in plain words, and announce it with aria-live.
- Use the correct input types (email, tel), inputmode, autocomplete and maxlength so mobile keyboards and autofill help the user.
- Show format hints, mark required fields, keep the entered data after an error, and move focus to the first invalid field.
- Do not show errors before the user has finished typing in the field.

Security

- Always repeat the validation on the server, because client-side checks can be bypassed with developer tools or direct requests.
- Escape or sanitise user input before displaying it, to prevent cross-site scripting (XSS), and use parameterised queries to prevent SQL injection.
- Use HTTPS, CSRF tokens, and rate limiting or CAPTCHA to protect the form from misuse.

- Keep regular expressions simple and tested, since badly written ones can reject valid users or become slow on long input.

Question 3: Box Model, Positioning and Layout Breaking

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

SOLUTION:

(a) The CSS box model and its components

The browser treats every HTML element as a rectangular box made of four layers, from the inside to the outside:

Component	Description
Content	The text or image itself. Its size is set by width and height.
Padding	Transparent space between the content and the border. It takes the background colour of the element.
Border	The line around the padding and content, set with border.
Margin	Transparent space outside the border that separates the element from its neighbours.

With the default box-sizing: content-box, the width of the box on the page is width + padding + border. For example, width: 250px with padding: 20px and border: 2px gives 250 + 40 + 4 = 294px. With box-sizing: border-box, the declared width already includes padding and border, so the box stays 250px wide.

(b) Effect of CSS positioning on layout structure

Value	Effect on the layout
static	Default. The element stays in the normal document flow, and top, left have no effect.
relative	The element is shifted from its normal place, but its original space is still reserved, so neighbours do not move. It is also used as the reference for absolute children.
absolute	The element is removed from the flow and placed relative to the nearest positioned ancestor. Other elements ignore it, so overlaps and gaps can occur.
fixed	The element is removed from the flow and stays at a fixed place in the viewport while scrolling, for example a top bar.
sticky	Acts as relative until a scroll position is reached, then sticks like fixed, for example a sticky header.

float also takes elements out of normal flow, so parents collapse and following elements wrap around the floated box unless the float is cleared.

(c) Possible causes of layout breaking

- Fixed widths in pixels: a 900px container is wider than a 420px phone screen, so the page scrolls sideways.
- Box model miscalculation: with content-box, a 250px sidebar becomes 294px and a 650px column becomes 694px. Together they are 988px, which is wider than the 900px container, so the second column drops below the first.
- Floats that are not cleared and layouts that depend on exact widths.

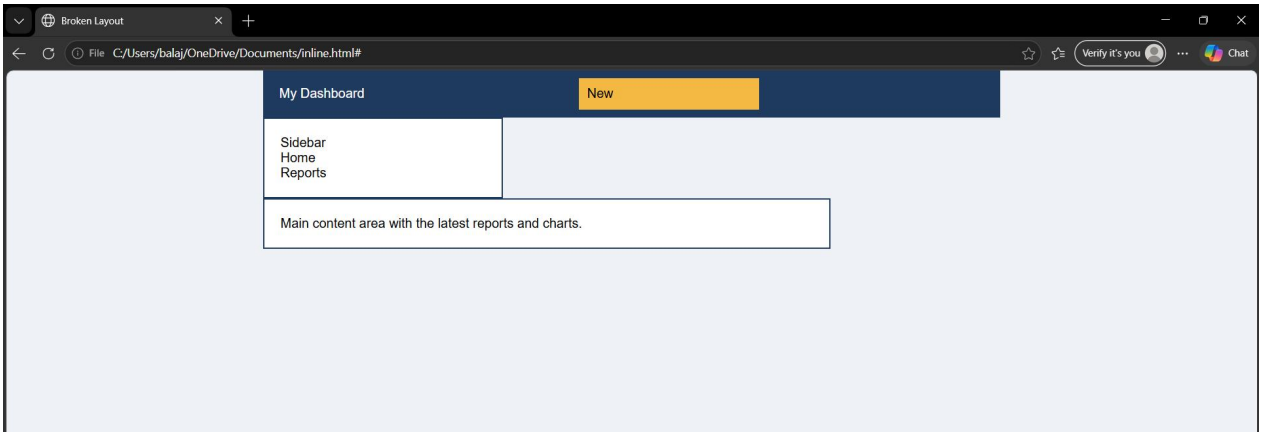
- Absolute positioning with fixed offsets, such as left: 700px, which pushes an element outside a narrow screen.
- Unbounded content: large images, wide tables or long words without max-width or overflow handling.
- Missing viewport meta tag or media queries, so the page is never adjusted for small screens.

The following page shows these faults. It was opened on a 420px wide screen.

FAULTY HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Broken Layout</title>
<style>
body { margin: 0; font-family: Arial, sans-serif; background: #eef2f7; }
.container { width: 900px; margin: 0 auto; }
.header { background: #1f3a5f; color: #fff; padding: 20px; }
.sidebar { float: left; width: 250px; padding: 20px; background: #fff;
          border: 2px solid #1f3a5f; }
.content { float: left; width: 650px; padding: 20px; background: #fff;
          border: 2px solid #1f3a5f; }
.badge { position: absolute; left: 700px; top: 10px; width: 200px;
        background: #f4b942; padding: 10px; }
</style>
</head>
<body>
<div class="container">
  <div class="header">My Dashboard</div>
  <div class="sidebar">Sidebar<br>Home<br>Reports</div>
  <div class="content">Main content area with the latest reports and charts.</div>
</div>
<div class="badge">New</div>
</body>
</html>
```

OUTPUT:



The page is 920px wide on a 420px screen, the columns have wrapped under each other, and the badge is pushed outside

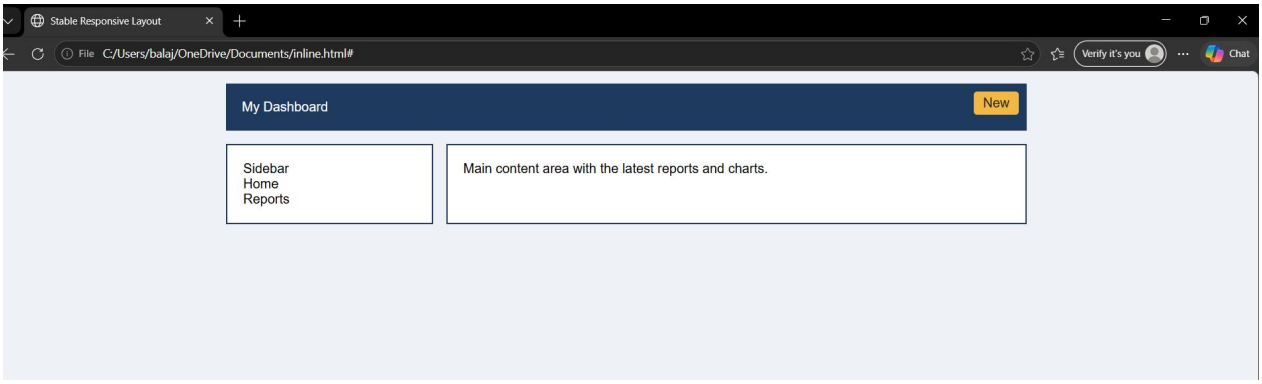
(d) Using media queries to resolve the problems

A media query applies CSS only when a condition, such as the screen width, is true. The fix is to write a simple one-column layout for small screens and use min-width queries to add the side-by-side layout when there is enough room. Together with border-box, max-width, Flexbox and a positioned parent for the badge, the layout stays stable at every width.

CORRECTED HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Stable Responsive Layout</title>
<style>
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; background: #eef2f7; }
.container { max-width: 1000px; margin: 0 auto; padding: 1rem;
            display: flex; flex-direction: column; gap: 1rem; }
.header { position: relative; background: #1f3a5f; color: #fff; padding: 1.2rem; }
.badge { position: absolute; top: 0.6rem; right: 0.6rem;
         background: #f4b942; color: #222; padding: 0.3rem 0.7rem;
         border-radius: 4px; }
.box { background: #fff; border: 2px solid #1f3a5f; padding: 1.2rem; }
.main { display: flex; flex-direction: column; gap: 1rem; }  /* mobile: stacked */
.sidebar, .content { width: 100%; }
@media (min-width: 768px) {                          /* tablet / desktop */
  .main { flex-direction: row; }
  .sidebar { flex: 0 0 250px; }
  .content { flex: 1; }
}
</style>
</head>
<body>
<div class="container">
  <div class="header">My Dashboard <span class="badge">New</span></div>
  <div class="main">
    <div class="box sidebar">Sidebar<br>Home<br>Reports</div>
    <div class="box content">
      Main content area with the latest reports and charts.
    </div>
  </div>
</div>
</body>
</html>
```

OUTPUT:



(e) Best practices for a stable and responsive layout

- Add *, *::before, *::after { box-sizing: border-box; } at the top of every stylesheet.

- Use relative units (%, rem, fr) and max-width instead of fixed pixel widths.
- Build page structure with Flexbox or Grid, not floats, and use position: absolute only for small elements inside a position: relative parent.
- Follow a mobile-first approach with min-width queries, and place breakpoints where the content starts to break.
- Make media fluid with img { max-width: 100%; height: auto; }, and use overflow-x: auto or overflow-wrap: anywhere for wide tables and long words.
- Always include the viewport meta tag, and test the page by resizing the window and using device mode in browser developer tools.

Question 4: DOM and Dynamic Content Updates

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

SOLUTION:

(a) Concept and role of the Document Object Model

When a browser loads a page, it converts the HTML into a tree of objects called the Document Object Model (DOM). The document object is the root, and the tree continues as html, then body, then elements such as ul, then li, then text nodes. Every element, attribute and piece of text is a node that JavaScript can read and change.

- Bridge between HTML and JavaScript: it gives scripts a standard API (document, Element, Node) to access the page.
- Dynamic pages: changing a node updates only that part of the screen, so the rest of the page is not reloaded.
- Interactivity: events such as click and input are attached to DOM nodes.
- Base for modern applications: data from fetch() can be turned into DOM elements, which is how single-page applications and live feeds work.

(b) Manipulating the DOM to update content dynamically

Task	Common DOM methods and properties
Select elements	getElementById(), querySelector(), querySelectorAll()
Create elements	createElement(), createTextNode()
Change content	textContent, innerHTML, setAttribute(), classList.add/remove/toggle(), style
Insert elements	appendChild(), append(), prepend(), insertBefore()
Remove elements	element.remove(), removeChild()
React to the user	addEventListener("click", handler)

A dynamic update normally follows five steps:

1. Select the container that will change.
2. Create a new element with document.createElement().
3. Set its text, attributes and classes.
4. Insert it into the container, or remove an existing element.
5. Respond to events, so the change happens when the user clicks, types, or when fetch() returns data.

(c) Example

The wishlist below adds and removes items and updates the item count without reloading the page. New items are created with createElement() and textContent, and one listener on the list handles every delete button (event delegation).

HTML, CSS & JAVASCRIPT CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Dynamic Wishlist</title>
<style>
body { font-family: Arial, sans-serif; background: #eef2f7; display: flex;
       justify-content: center; padding: 30px; }
.wrap { background: #fff; width: 360px; padding: 18px; border-radius: 8px;
        box-shadow: 0 1px 4px rgba(0,0,0,.2); }
h2 { color: #1f3a5f; margin-bottom: 10px; }
.row { display: flex; gap: 8px; }
#item { flex: 1; padding: 8px; border: 1px solid #999; border-radius: 4px; }
#addBtn { padding: 8px 14px; border: 0; border-radius: 4px; background: #1f3a5f;
          color: #fff; cursor: pointer; }
ul { list-style: none; margin-top: 12px; }
li { display: flex; justify-content: space-between; align-items: center;
     padding: 8px 10px; margin-bottom: 6px; background: #e8f0fa;
     border-radius: 4px; }
.del { border: 0; background: none; color: #c0392b; font-size: 1.3rem;
       cursor: pointer; }
#count { margin-top: 10px; font-weight: bold; color: #1f3a5f; }
</style>
</head>
<body>
<div class="wrap">
  <h2>My Wishlist</h2>
  <div class="row">
    <input type="text" id="item" placeholder="Add an item" aria-label="New item">
    <button id="addBtn">Add</button>
  </div>
  <ul id="list">
    <li><span>Headphones</span>
      <button class="del" aria-label="Remove Headphones">&times;</button></li>
    <li><span>Smart Watch</span>
      <button class="del" aria-label="Remove Smart Watch">&times;</button></li>
    <li><span>Notebook</span>
      <button class="del" aria-label="Remove Notebook">&times;</button></li>
  </ul>
  <p id="count"></p>
</div>
<script>
const list = document.getElementById("list");
const input = document.getElementById("item");
const count = document.getElementById("count");

function updateCount() { count.textContent = "Items: " + list.children.length; }

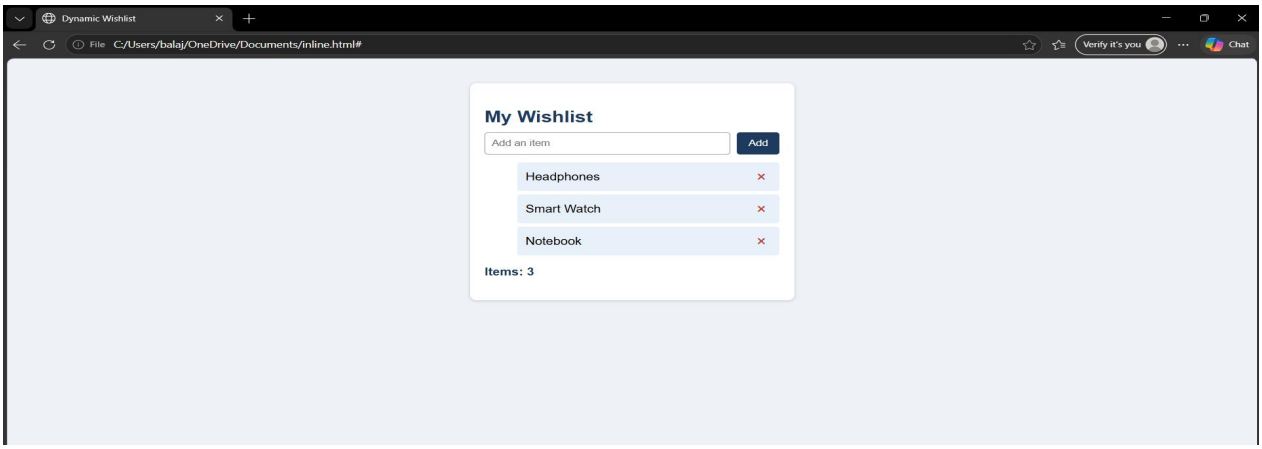
function addItem() {
  const text = input.value.trim();
  if (text === "") { input.focus(); return; }
  const li = document.createElement("li");          // create a new node
  const span = document.createElement("span");
  span.textContent = text;                // textContent: safe from HTML injection
```

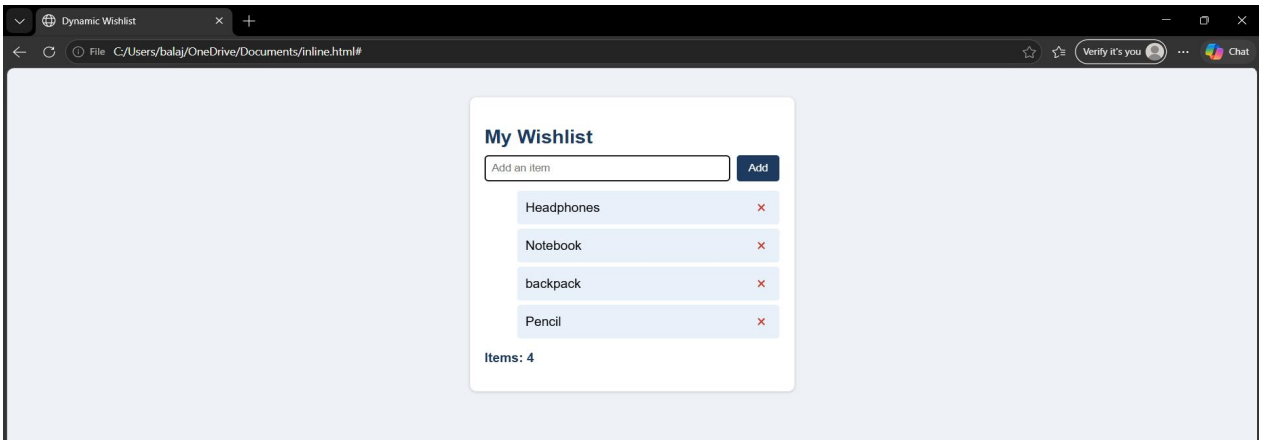
```
  const btn = document.createElement("button");
  btn.className = "del";
  btn.textContent = "\u00D7";
  btn.setAttribute("aria-label", "Remove " + text);
  li.append(span, btn);
  list.appendChild(li);                    // insert into the live DOM
  input.value = "";
  input.focus();
  updateCount();
}

document.getElementById("addBtn").addEventListener("click", addItem);
input.addEventListener("keydown", function (e) {
  if (e.key === "Enter") addItem();
});

// Event delegation: one listener on the list handles every delete button
list.addEventListener("click", function (e) {
  if (e.target.classList.contains("del")) {
    e.target.parentElement.remove();          // remove the node
    updateCount();
  }
});
updateCount();
</script>
</body>
</html>
```

OUTPUT:





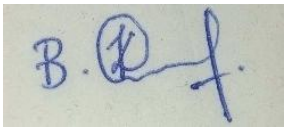
d) Improvements to enhance performance and user experience

- Reduce reflows: build many new nodes in a DocumentFragment or an array and insert them once, instead of updating the page many times.

- Event delegation: one listener on the parent, as used above, instead of one listener per item.
- Cache selectors: store the result of getElementById() in a variable instead of searching the DOM again and again.
- Use `textContent` instead of `innerHTML` for user text, which is faster and prevents cross-site scripting.
- Change classes, not styles: classList.toggle() is cleaner than setting many style properties.
- Control event frequency: debounce or throttle input, scroll and resize handlers.
- Large lists: use pagination, lazy loading or virtual scrolling so the DOM stays small.
- User experience: show a loading indicator and an error message for fetch() calls, use aria-live so screen readers announce updates, and keep focus on a sensible element after adding or removing items.

Code of Conduct:

I B KIRTHANAA 192324244 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: B KIRTHANAA

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand